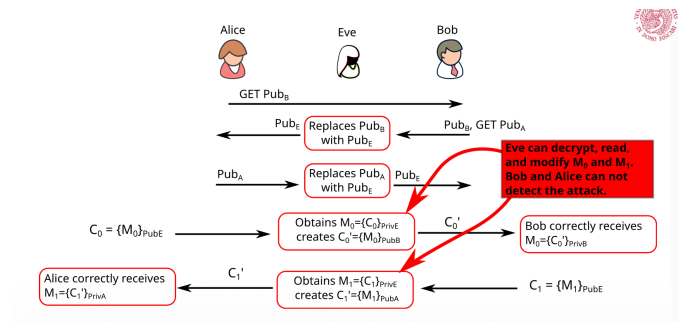
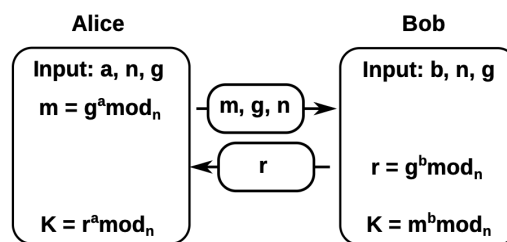


11-5



Diffie-Hellman protocol

DH è un protocollo che permette a Alice e Bob per negoziare una chiave simmetrica segreta senza scambiarsi informazioni segrete.



- Alice prende n un numero primo, a casuale e g radice prima modulo n .
- Bob genera un numero casuale b
- a e b sono numeri segreti, non lasciano mai Bob e Alice
- Alice manda m, g, n a Bob
- Bob manda r ad Alice
- la chiave generata è $K = g^{a,b} \text{mod}(p)$

dimostrazione

$$K' = r^a \text{mod}_n$$

$$K'' = m^b \text{mod}_n$$

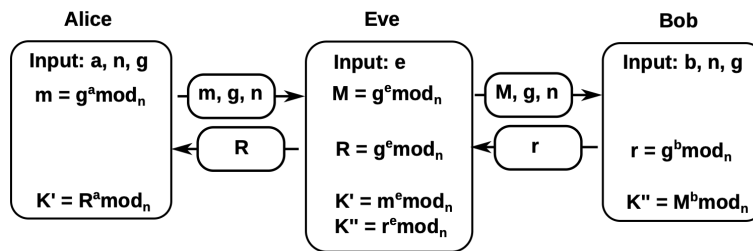
dobbiamo verificare che $K' = K''$

D

se Eve riesce a intercettare passivamente il traffico, sa che m, g, n e r ma per la durezza del problema logaritmico discreto non può ottenere ne a ne b .

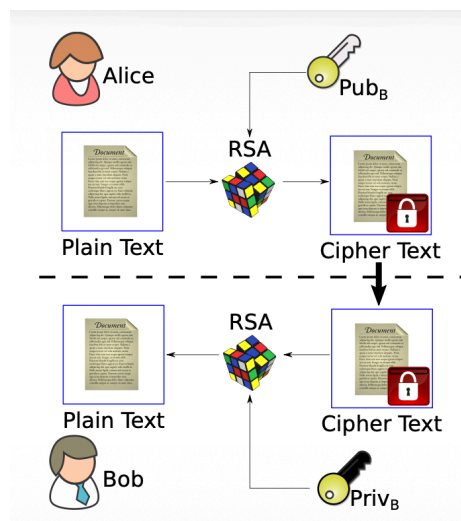
K è segreto e può essere usato da Alice e Bob come chiave per un algoritmo a chiave simmetrica, generato in maniera indipendente da Alice e Bob, non è mai trasmesso nel canale.

Active MiTM Attack



Eve in questo caso può leggere e modificare il traffico tra Alice e Bob, quindi DH può garantire la confidenzialità solamente se è usato con un altro strumento che permetta di garantire l'autenticazione.

public key encryption



- nella public key encryption ciò che è encrypted con la chiave pubblica può essere decriptato solamente con la corrispondente chiave privata
- quindi se Alice encrypt il messaggio M con la chiave pubblica ottenendo C e poi cancella il messaggio M non potrà riottenere M da C anche se è stata lei a fare l'encryption

abbiamo bisogno di una funzione $f : D \rightarrow R$ con un parametro t che abbia 3 caratteristiche.

$f : D \rightarrow R$ easy

$f : R \rightarrow D$ easy conoscendo t

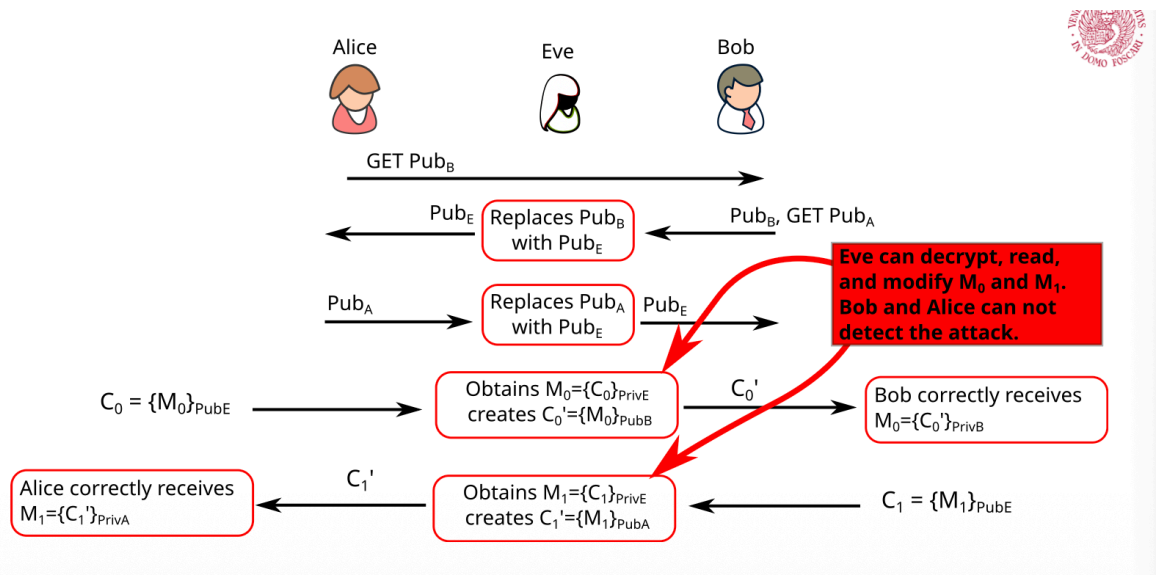
$f : R \rightarrow D$ impossibile senza conoscere t



purtroppo non esiste una funzione matematica simile. In generale le funzioni matematiche sono invertibili o no, ancora dobbiamo appoggiarci a un'approssimazione che lo rende computazionalmente impossibile

PK protocol

1. $A \rightarrow B : GET \text{ Pub}_B$
2. $B \rightarrow A : \text{Pub}_B, GET \text{ Pub}_A$
3. $A \rightarrow B : \text{Pub}_A$
4. $A \rightarrow B : \{M_0\}_{\text{Pub}_B}$
5. $B \rightarrow A : \{M_1\}_{\text{Pub}_A}$



la differenza più grande tra public key cryptography e Simmetric key cryptography è che la prima necessita di un canale autentificato, integro e segreto, mentre l'ultima necessita solamente di un canale **autentificato e integro**

digital signature

il concetto chiave è che nella PKE il processo di encrypting è reversibile, il messaggio di Alice se è criptato con la chiave privata, è decryptabile con la chiave pubblica:

se $C = \{M\}_{\text{Priv}_A}$ allora $M = \{C\}_{\text{Pub}_A}$

questo permette però a chi decrypta il messaggio con la chiave pubblica a essere sicuro che quel messaggio proviene da Alice che è l'unica a conoscere Priv_A la sua chiave privata.



questo processo di FIRMA DIGITALE l'autenticazione dei dati

anche in questo caso l'attacco di tipo MiTM è ancora possibile, prima di verificare una firma digitale Alice ha bisogno di ricevere la chiave di Bob, se Eve attacca rimpiazzerà entrambe le chiavi e le firme.

RSA è più oneroso nell'encryption, il server che decrypta il traffico ha bisogno di più risorse del client che le crypta.



generalmente RSA è usato solo per scambiarsi la chiave segreta e poi per lo scambio di dati è usato encrypting di chiave simmetrica e HMAC

esempio di scambio chiavi

1. $A \rightarrow B : \text{PubA}$
2. $B \rightarrow A : \text{PubB}$
3. $A \rightarrow B : \{N_1\}_{\text{PubB}}$
4. $B \rightarrow A : \{n_2\}_{\text{PubA}}$
5. $K_{AB} = H(N_1, N_2); A \rightarrow B : \{M\}_{K_{AB}}$

esempio di protocollo di integrità e autenticazione

1. $A \rightarrow B : M, D = H(M), C\{D\}_{\text{PrivA}}$
2. Bob allora:
 - a. computa $D' = \{C\}_{\text{PubA}}$
 - b. controlla che $D' = D$ per verificare che la firma sia valida e quindi il messaggio autenticato
 - c. computa $D'' = H(M)$
 - d. controlla se $D'' = D$ allora il digest è valido quindi il messaggio è integro

public keys

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
mQINBFJ03JYBEADtKuOxyMibiwnKZw6rD7gjmRi4UTaIA3v9ro0gUU0r51g7Eu1p
DWjUjZ4ReXkZyQ20YXK0ozV5CxMEsiOcN92tgAhwK1fgqZrz2/N4NJdbzY/Y56zl
[...]
C6oR0TqUsO3sZRj6ITJyaZGrY1y/RrziH9xtLBUqyVTmeAx+B5GWQgvoaPFRsbMy
/r3xp+vOU48NRu0dSt/TowN1p1QDdlfeDYBrbeGbFzY+MiFaX0DrhHDwRA==
=pilJ
-----END PGP PUBLIC KEY BLOCK-----
```

è un file contenente stringhe casuali, è impossibile deciderne il contenuto quindi non può contenere informazioni utili, questo non conferma che chi ha dice di aver generato la chiave è colui che l'ha effettivamente generata.

key fingerprint

Alice genera le sue chiavi pubblica l'hash della chiave pubblica ovunque in internet: il suo sito, la firma della sua email. Quando Bob riceve la chiave, può controllare che il fingerprint sia lo stesso che Alice sponsorizza in giro

key servers

in internet esistono dei server di chiavi, dove caricare le proprie chiavi pubbliche, assieme a metadati

web of trust

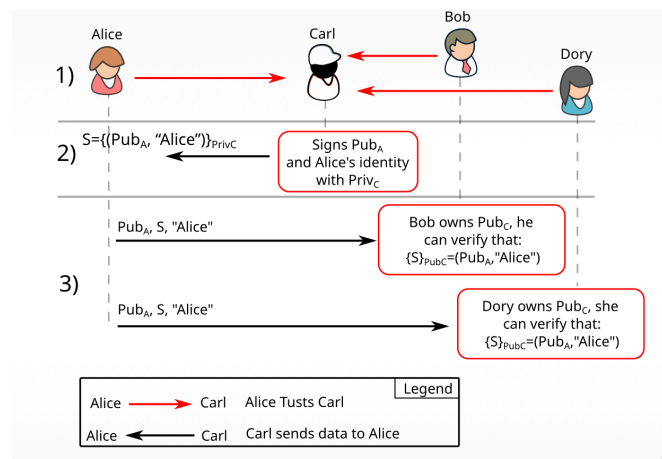
è un social network di contratti, in cui i partecipanti certificano l'identità degli altri.

trust: nel contesto del PKE la trust è una relazione direzionale; se Alice si fida di Carl vuol dire che Alice conosce già la chiave pubblica di Carl e crede che certifica sia vero.

Carl da a Alice una key signature

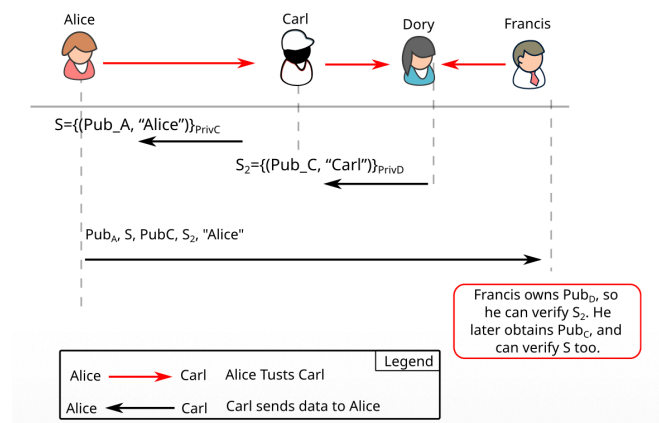
$$S = \{(Pub_A, "Alice")\}_{Priv_C}$$

Carl cripta entrambe Pub_A e l'identità di Alice. Chiunque conosca Pub_C e si fida di Carl può conoscere la chiave pubblica di Alice.

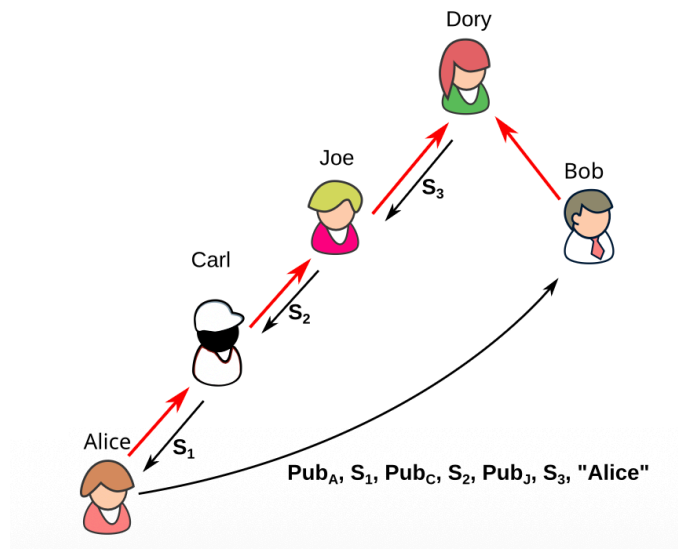


nel momento in cui Alice ha una chiave firmata, Carl non è più utile, di fatto Alice può utilizzare la sua chiave firmata con chiunque si fida di Carl

La trust si può muovere anche per percorsi più lunghi.



si crea quindi un **trust path**



public key infrastructure

è un infrastruttura dove c'è uno o più party che hanno la fiducia di tutti (a trusted third party) chiamato anche Certification Authority (CA).

Una chiave firmata con dei metadati è chiamato certificato:

1. Alice genera **PubA** e **PrivA**
2. manda una CSR (Certificate Signing request) firmata con PrivA:

$$CSR = (\text{PubA}, "Alice", \{\text{PubA}, "Alice"\}_{PrivA})$$
2. la CA usa PubA per decriptare il file e verificare la firma, quindi la CA sa che Alice ha PrivA
3. La CA restituisce un certificato valido : la CSR con la firma della CA aggiunta:

$$Cert = (CSR, metadata, \{(CSR, metadata)\}_{PrivCA})$$

I certificati utilizzano un formato standard, chiamato X.509 e contengono molti metadati

Issuer Name
Cert. Serial Number
Validity Period
Subject Name
Subject Public Key
Certificate Signature by the CA

poichè il processo di firma con chiave pubblica è computazionalmente oneroso, la firma del certificato è in realtà la firma di un hash del certificato stesso, deve anche includere il nome della suddetta funzioni di hash.

certificato

$Data = (\text{PubA}, "Alice", \text{Issuer}, \text{validity} \dots)$

$D = H(Data)$

$\text{Cert} = (\text{Data}, \{D\}_{PrivCA})$

Per validare un certificato Bob:

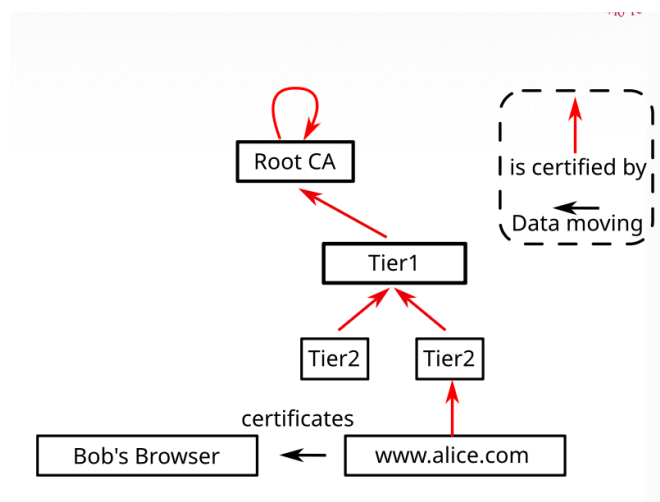
- controlla che il certificato sia a nome Alice,
- controlla che il certificato non sia scaduto.
- Controlla di fidarsi della CA e se possiede la sua chiave pubblica
- Decrypta la firma con la chiave pubblica della CA e ottiene il digest D poi hasha il messaggio per ottenere D' e controlla la corrispondenza $D = D'$

I certificati sono molto usati per i domini di rete. Le CA hanno delle gerarchie e si può creare una catena di fiducia, ogni CA riceve un certificato da una CA di livello più alto.

questo è un esempio di asymmetric trust, alla fine il browser conosce la chiave pubblica del server che usa quel dominio in questo modo il browser può ricevere dati da quel server,



la comunicazione è sicura solamente in **una** direzione



le CA hanno una gerarchia e la CA di ogni livello è caertificata dalla CA di un livello superiore; la CA alla radice **certifica se stessa**. Questo accade solo per poche root CAs

certificate Revocation List

è una lista di certificati che sono stati revocati dalla CA , un CRL è firmato dalla chiave della CA stessa e pubblicato dalla CA



il sistema delle PKI riduce il numero di entità di cui fidarsi, non qualsiasi dominio visitato, ma solo un piccolo numero di CAs in quanto la fiducia è organizzata ad albero

se violata?

in generale ci fidiamo delle CAs, ma nel caso una venisse crackata le conseguenze sarebbero catastrofiche: se quella CA ha una catena fino a una root CA può rilasciare certificati per qualsiasi cosa persino Google.com